EMBEDDED / ELECTRONICS / ROBOTICS / CASE STUDY

CUSTOM AUTONOMOUS DRONE

CASE STUDY

01

Reconstructing a past drone build into an evidence pack that separates tested fact from memory.

● EXISTING BUILD / EVIDENCE TO CURATE

THE BUSINESS CONTEXT

An engineer built an autonomous aircraft some time ago across mechanical design, boards, firmware and a phone application. The artefacts survive but are scattered across folders and notes. Anyone assessing the work needs to know what was actually flown or measured and what only existed as design intent.

WHO IS INVOLVED AND WHAT THEY NEED

01	**Original builder**	Recover the real history without overstating what the hardware proved.
02	**Technical reviewer**	Needs traceable evidence for each claim about flight and control behaviour.
03	**Hiring manager**	Wants to judge embedded and systems depth from artefacts rather than narrative.

SCOPE AND HONEST LIMITS

⚠ Recovered code and diagrams do not prove airworthiness or repeatable flight.

⚠ Reconstructed assumptions are labelled and may be wrong.

⚠ Lost logs cannot be recreated so some claims will stay unsupported.

⚠ Nothing here is a measured result until the evidence exists to show it.

CURRENT WORKFLOW AND FAILURE POINTS

01

EXISTING BUILD / EVIDENCE TO CURATE

HOW THE WORK RUNS TODAY

1 TODAY
Scattered artefacts
Source, schematics, CAD, photos and notes sit in separate folders with no index.

→

2 TODAY
Memory based narrative
The build story is written from recollection and tends to blend plan with outcome.

→

3 TODAY
Informal test notes
Bench and flight observations live in chat logs or loose files without dates.

4 TODAY
Unversioned firmware
Several firmware variants exist and it is unclear which one flew.

→

5 TODAY
Selective showcase
Best photos and clips are chosen and failures quietly dropped.

→

6 TODAY
One off write up
A summary page is produced once and cannot be checked or rerun.

WHERE IT BREAKS

Unclear provenance	A claim cannot be tied back to a file or a dated test.
Plan blurred with result	Simulation and concept work read as if flown on hardware.
Lost failure history	Fixes and failed builds explain the design but are not recorded.
Unverifiable controller	The preserved control code has no regression test to check it still behaves.

REDESIGNED WORKFLOW AND DECISION AUTHORITY

01

EXISTING BUILD / EVIDENCE TO CURATE

THE REDESIGNED WORKFLOW

1 SYSTEM

Inventory artefacts

List every recovered file with its type and date and where it came from.

→

2 PERSON

Classify each claim

Mark every statement as flown, bench tested, simulated or conceptual.

→

3 SYSTEM

Reconstruct interfaces

Rebuild subsystem requirements and interfaces from source and schematics.

↓

4 SYSTEM

Run software in the loop

Exercise the preserved controller in simulation and log assumptions made.

→

5 PERSON

Rights and safety review

Check publication rights and sensitive material before anything is released.

→

6 SYSTEM

Publish traceable case

Release the architecture map and verification inventory with gaps stated.

SYSTEM STEP PERSON DECISION

WHERE AUTHORITY STAYS WITH A PERSON

Evidence classification of a claim	ORIGINAL BUILDER	Only the builder can say what was actually flown or measured.
Acceptance of a reconstructed assumption	TECHNICAL REVIEWER	An independent view stops convenient assumptions becoming facts.
Public release of files	ORIGINAL BUILDER	Rights and sensitive material need a named person to take responsibility.

ARCHITECTURE, DATA AND CONTROLS

01

EXISTING BUILD / EVIDENCE TO CURATE

SYSTEM ARCHITECTURE

1	**Artefact store**	Versioned copies of source, schematics, CAD, media and notes with hashes.
2	**Evidence register**	Each claim linked to artefacts and a status of flown or bench or simulated.
3	**Architecture map**	Subsystem diagram tracing power, firmware, control and application interfaces.
4	**Simulation harness**	Software in the loop runs of the preserved controller with fixed seeds.
5	**Publication pack**	README and curated media generated from the register with gaps visible.

CORE DATA ENTITIES

- **Artefact**
- **Claim**
- **Evidence link**
- **Subsystem**
- **Test record**
- **Assumption**

RISKS AND THE CONTROL FOR EACH

Simulation presented as flight →	Every claim carries a status field and the case study shows only what its evidence supports.
Wrong firmware version cited →	Each test record names the file hash of the firmware and board revision involved.
Sensitive material released →	Nothing is published until the builder signs off a rights and safety checklist.
Gaps hidden by tidy narrative →	Missing evidence is listed openly in a gaps section of the inventory.

EVALUATION AND DELIVERY PLAN

01

EXISTING BUILD / EVIDENCE TO CURATE

HOW SUCCESS IS MEASURED

Claim traceability	Share of case study claims that link to at least one dated artefact.
Evidence status split	Count of claims by status so flown and simulated work stay separate.
Open gaps	Number of claims with no supporting artefact after review.
Rerun success	Whether an independent person can rerun the software in the loop tests.
Reviewer queries	Questions a reviewer raises that the register could not answer.

TEST PLAN

1. **Pick ten claims at random and trace each to a file and a dated record.**
2. **Rerun the preserved controller in simulation from a clean machine and compare logs.**
3. **Check that no media item is labelled as flight footage without a matching log.**
4. **Have a second person find any statement lacking a status label.**

DELIVERY PLAN

01 ALIGN	02 UNDERSTAND	03 DESIGN	04 DELIVER	05 LAUNCH
Agree the claims to be tested and what counts as acceptable evidence for each.	Recover and inventory every artefact and note what is missing.	Define the register schema and the software in the loop test plan.	Build the register and the regression tests and write up the architecture.	Run the rights review and publish the pack with limits stated.
DECISION Which claims enter scope	**DECISION** What counts as lost evidence	**DECISION** Status labels and test design	**DECISION** Which assumptions are accepted	**DECISION** What is released publicly

DELIVERABLES

- Repository README and architecture map
- Evidence register with claim status
- Software in the loop regression tests
- Curated media pack with captions
- Verification inventory and gaps list